



Fundamentos de Front-End com React

Autoria: Daniel Xavier

Introdução

Esta disciplina tem como objetivo introduzir, de forma estruturada e progressiva, os principais fundamentos do desenvolvimento front-end moderno utilizando a biblioteca React. Ao longo de seis aulas, construímos uma base sólida que prepara o aluno para atuar com segurança em projetos reais, dominando tanto os aspectos técnicos quanto os desafios práticos do mercado.

Começamos com uma visão ampla do ecossistema front-end, destacando as transformações tecnológicas que levaram à adoção de frameworks como React. Analisamos o papel do JavaScript moderno, a evolução das interfaces web e a necessidade crescente de aplicações mais interativas, reativas e escaláveis.

Na segunda aula, partimos para a prática com a criação de um projeto React utilizando o Create React App. Exploramos a estrutura de arquivos, o conceito de componentes, JSX e o funcionamento do DOM virtual. Essa base nos permitiu compreender como o React organiza e atualiza a interface com eficiência.

Em seguida, aprofundamos o uso de `useState` e `useEffect` para consumir APIs REST, tratar estados de loading e erro, e exibir dados de forma dinâmica. Esse conhecimento foi essencial para preparar o terreno para aplicações conectadas a dados reais.

Na aula sobre autenticação, tratamos da segurança no front-end, com foco em JWT, armazenamento de tokens, proteção de rotas e boas práticas para manter sessões autenticadas com responsabilidade e clareza arquitetural.

Com esse repertório, entramos no projeto guiado dividido em duas partes: construímos um mini app funcional de catálogo de filmes, aplicando todos os conceitos anteriores. Utilizamos chamadas de API, busca, renderização condicional, responsividade e roteamento com `react-router-dom`, organizando o código em páginas e componentes reutilizáveis.

Finalizamos com uma aula voltada para entrevistas técnicas e desafios práticos, preparando o aluno para processos seletivos. Discutimos tipos de entrevistas, como se preparar, o que é esperado e como aplicar o que foi aprendido neste módulo na vida real.

Ao concluir esta disciplina, o aluno está apto a desenvolver aplicações com React, estruturando o código de forma profissional, consumindo APIs, aplicando boas práticas de autenticação e se posicionando com competência em entrevistas e desafios de mercado. Este é o ponto de partida para uma carreira sólida no front-end moderno.

Bons estudos!

Capítulo 1: Introdução ao Front-End Moderno

Seja bem-vindo ao curso de Fundamentos de Front-End com React. Antes de irmos para o código, é importante construirmos uma base sólida sobre o que é o front-end moderno, como ele evoluiu ao longo do tempo e quais tecnologias compõem esse ecossistema atualmente.

Sobre mim e os objetivos do curso

Sou desenvolvedor front-end e tech lead com experiência tanto em desenvolvimento quanto em liderança técnica e gestão de engenharia. Atuei por anos no mercado nacional, empreendi e atualmente trabalho como desenvolvedor no mercado internacional. Esse curso foi estruturado com base nessa trajetória e tem como objetivos principais:

- Ensinar React de forma prática, moderna e conectada com o mercado;
- Ajudar a conectar a mentalidade de back-end com o front-end, promovendo aplicações mais equilibradas;
- Preparar você para entrevistas e desafios técnicos reais;
- Estimular a visão de produto, indo além do código.

Premissas para acompanhar o curso

O curso assume que você já possui alguns conhecimentos fundamentais:

1. Lógica de programação;
2. Experiência com desenvolvimento, especialmente em Java;
3. Entendimento do funcionamento de aplicações web;
4. Noção sobre comunicação assíncrona;
5. Hábito de consultar documentação técnica.

Evolução do Front-End

Para entender o que é o front-end moderno, precisamos olhar para sua evolução. Tudo começa nos anos 1990 com a criação do HTML (1991) e do CSS (1996), que trouxeram a separação entre estrutura e estilo.

Na virada para os anos 2000, o JavaScript e o Ajax trouxeram interatividade para as páginas. A seguir, surgiram as SPAs (Single Page Applications), que permitem atualizações de conteúdo sem recarregar toda a página.

Com o tempo, vieram os frameworks modernos como React, Vue e Angular, acompanhados por ferramentas de build, novas versões do ECMAScript e um ecossistema mais modular e robusto.

Hoje, falamos de front-end moderno quando usamos:

- Server-side rendering;
- TypeScript;
- Metaframeworks como Next.js;
- Ferramentas e abordagens que transformam o front-end em uma plataforma completa de desenvolvimento.

Fundamentos das tecnologias

HTML é a estrutura de tudo. É uma linguagem de marcação, não de programação. Define o que é cada parte da página.

CSS estiliza o HTML. Apesar de simples na teoria, na prática pode ser desafiador. Importante lembrar: CSS não existe sem HTML.

JavaScript é a linguagem de programação que traz lógica, interatividade e dinamismo. É baseada em objetos e interpretada em tempo de execução.

React é uma biblioteca focada em componentes reutilizáveis. Usa JSX, que mistura HTML e JavaScript. Cada componente tem uma responsabilidade específica.

Next.js é um metaframework baseado em React. Garante organização de rotas, renderização no servidor e performance. É uma solução mais completa.

Tailwind CSS é um framework utilitário de CSS. Trabalha com classes que aplicam estilos específicos. Traz produtividade, especialmente com a abordagem mobile first.

TypeScript adiciona tipagem estática ao JavaScript. É muito útil para evitar erros e melhorar a manutenção. Em tempo de execução, continua sendo JavaScript.

Node.js permite executar JavaScript no servidor. É a base para rodar nossas aplicações e usar o gerenciador de pacotes NPM.

Boas práticas e conceitos essenciais

- Documentação é sempre a principal referência. Use e consulte sempre que precisar.
- Componentização é a base do desenvolvimento moderno com React.
- Acessibilidade vem de boas escolhas de HTML. Usar os elementos corretos garante acessibilidade em grande parte dos casos.
- Performance é essencial, e no front-end também importa. Server-side rendering é uma das ferramentas para alcançá-la.
- Responsividade deve ser pensada desde o início. O Tailwind ajuda nisso, mas é fundamental entender CSS.

Ambiente de desenvolvimento

Para seguir com o curso:

- Instale o **Node.js** (versão LTS);
- Use o **NVM** para gerenciar versões do Node;
- Utilize o **Visual Studio Code**, um editor leve e eficiente.

Esse primeiro contato tem como objetivo dar clareza sobre onde estamos pisando. O front-end moderno é poderoso, flexível e cheio de possibilidades. Agora que você tem uma visão geral, seguimos aprofundando cada um desses tópicos nas próximas aulas.

Capítulo 2: Primeiros Passos com React

Neste capítulo, vamos iniciar nossa jornada prática com React. A ideia é sair da teoria e começar a construir de fato. Vamos abordar os conceitos essenciais para entender como o React funciona, como montar a estrutura inicial do projeto e como interpretar a interface e o comportamento básico de uma aplicação React.

Criando o projeto com Create React App

O ponto de partida mais comum é usar o Create React App, uma ferramenta que abstrai toda a configuração inicial. Ela já traz tudo pronto para iniciar o desenvolvimento sem precisar configurar Webpack, Babel e outros detalhes do ecossistema.

Ao criar o projeto com o `npx create-react-app`, é gerada uma estrutura de pastas e arquivos que inclui o `public/`, onde ficam arquivos estáticos como o `index.html`, e o `src/`, onde fica o código fonte da aplicação. O ponto de entrada é o `index.js`, que renderiza o componente principal App dentro do elemento com `id="root"` no `index.html`.

O React trabalha com o conceito de DOM virtual. Isso significa que ele mantém uma representação em memória da estrutura da página, e só atualiza o DOM real quando há alguma mudança. Isso torna o desempenho muito mais eficiente.

JSX e a estrutura dos componentes

Uma das principais características do React é o uso de JSX, uma extensão de sintaxe que permite escrever HTML dentro do JavaScript. Apesar de parecer HTML, o JSX é convertido para funções JavaScript que criam elementos React.

Um componente em React é, basicamente, uma função que retorna JSX. Ele pode receber propriedades (props) que são parâmetros utilizados para tornar o componente reutilizável e dinâmico.

Importante lembrar que, ao usar JSX, não se pode retornar mais de um elemento "solto". Sempre devemos envolver os elementos em um contêiner, como uma `div`, ou usar um fragmento (`<> </>`).

Entendendo o App.js e o fluxo da aplicação

No App.js, temos um exemplo clássico de um componente funcional. Esse componente é exportado como padrão e importado no index.js, onde é renderizado dentro da árvore do React. Esse fluxo é fundamental para entender como os componentes se relacionam.

Além disso, vale destacar que o React é declarativo. Isso significa que descrevemos o que queremos ver na tela, e o React cuida de atualizar a interface quando os dados mudam. Isso é diferente do modelo imperativo, onde precisamos dizer passo a passo o que deve acontecer.

Componentes reutilizáveis e boas práticas

Uma das grandes forças do React é a capacidade de criar componentes reutilizáveis. Para isso, devemos manter a responsabilidade de cada componente bem definida. Um componente deve fazer apenas uma coisa, e fazer bem.

Componentes podem ser compostos, ou seja, um componente pode importar e utilizar outros componentes. Isso ajuda a manter o código organizado e facilita a manutenção.

Outro ponto importante é o uso correto das props. Elas são essenciais para tornar os componentes reutilizáveis e dinâmicos, mas é importante entender que props são imutáveis. Um componente não deve tentar alterá-las.

Conclusão

Compreender a estrutura inicial de uma aplicação React, o papel do JSX, o fluxo entre os arquivos principais e o conceito de componentes são passos fundamentais para avançar com segurança. Ao longo do curso, vamos evoluir essa base com recursos mais avançados, mas tudo parte desses conceitos que exploramos neste capítulo.

A próxima etapa é explorar como manipular dados dentro dos componentes, interagir com o usuário e organizar o código de forma escalável. Seguimos!

Capítulo 3: Comunicação com APIs

Neste capítulo, vamos explorar um dos aspectos mais importantes no desenvolvimento de aplicações modernas com React: a comunicação com APIs. Este é o ponto de conexão entre o front-end e os dados que geralmente estão

armazenados em servidores ou sistemas externos. Vamos ver como buscar, exibir e tratar essas informações de maneira eficiente.

Entendendo o papel das APIs

API (Application Programming Interface) é um meio padronizado de comunicação entre sistemas. Em aplicações web, usamos APIs REST para realizar operações como buscar dados, criar, atualizar e excluir registros. No front-end, a comunicação com essas APIs é feita geralmente via requisições HTTP (GET, POST, PUT, DELETE).

React, por ser apenas uma biblioteca de interface, não possui opinião sobre como essa comunicação deve acontecer. Por isso, usamos ferramentas complementares, como a própria `fetch` da Web API ou bibliotecas como `Axios`, para realizar essas chamadas.

Trabalhando com o ciclo de vida

Como React é baseado em componentes, precisamos lidar com o momento correto para realizar a chamada à API. Numa função tradicional, usaríamos o `useEffect`, um hook que permite executar efeitos colaterais, como requisições HTTP, após a renderização do componente.

O `useEffect` é essencial porque evita chamadas desnecessárias e permite controlar quando e como o efeito ocorre. Normalmente, usamos ele com um array de dependências. Se esse array estiver vazio, o efeito ocorre apenas uma vez, ao montar o componente.

Tratando estados com `useState`

Para armazenar os dados vindos da API e controlar o que é exibido na tela, usamos o `useState`. Ele é um hook que nos permite criar variáveis reativas. Quando o valor do estado é alterado, o componente é re-renderizado com os novos dados.

Então, o fluxo mais comum é:

1. Criar uma variável de estado para armazenar os dados;

2. Utilizar `useEffect` para chamar a API e buscar os dados;
3. Atualizar o estado com os dados retornados;
4. Renderizar o conteúdo com base nesse estado.

Tratamento de erros e loading

Sempre que lidamos com chamadas externas, precisamos considerar falhas de rede ou problemas no servidor. Por isso, é boa prática ter um estado para controle de erro e outro para controle de loading.

- O estado de loading indica se os dados estão sendo carregados.
- O estado de error captura falhas na chamada da API e permite exibir mensagens amigáveis ao usuário.

Com isso, conseguimos melhorar a experiência do usuário, evitando interfaces quebradas ou vazias sem explicação.

Estrutura básica do componente com consumo de API

O componente React que consome uma API costuma ter:

- Importações dos hooks (`useState`, `useEffect`);
- Três estados: dados, loading e erro;
- Um `useEffect` que chama a API ao montar o componente;
- Um JSX que trata as três possibilidades (carregando, erro, dados).

Essa estrutura é um padrão que vamos repetir diversas vezes no curso, sempre que precisarmos interagir com dados externos.

Dicas práticas

- Sempre trate o JSON retornado com `response.json()` para transformar os dados em objeto JavaScript;
- Cuidado com APIs que exigem autenticação: é necessário enviar headers apropriados com tokens;
- Use ferramentas como Postman ou Insomnia para testar as requisições antes de implementá-las no código;

- Separar a lógica de consumo de API em arquivos auxiliares ajuda a manter o código limpo e organizado;
- Sempre valide os dados antes de exibi-los, para evitar erros de renderização.

Conclusão

Neste capítulo, aprendemos como integrar nossa aplicação React com APIs externas, utilizando `fetch`, `useEffect` e `useState`. Com isso, passamos a tornar nossas interfaces dinâmicas e conectadas com dados reais.

Na próxima etapa, vamos ver como organizar melhor nossos componentes, separar responsabilidades e estruturar a aplicação de forma mais profissional. Até lá!

Capítulo 4: Autenticação

Neste capítulo, vamos tratar de um tema central no desenvolvimento de aplicações: autenticação. Saber identificar quem está acessando a aplicação é essencial para garantir segurança, personalização e controle de acesso a funcionalidades. Vamos entender como esse processo funciona no front-end com React e quais boas práticas seguir.

Conceito de autenticação

Autenticação é o processo de verificar a identidade de um usuário. Em aplicações modernas, isso normalmente é feito através de um login com usuário e senha, que são validados no back-end. Quando a validação é bem-sucedida, o servidor retorna um token que representa aquela sessão autenticada.

Esse token é armazenado pelo front-end e enviado em chamadas subsequentes à API para que o back-end saiba quem está fazendo a requisição. Assim, é possível proteger rotas e recursos conforme a identidade e permissões do usuário.

JWT: JSON Web Token

O formato mais comum para representar sessões autenticadas é o JWT. Trata-se de um token que contém três partes codificadas em base64: cabeçalho (header), corpo (payload) e assinatura (signature).

O header informa o algoritmo de criptografia, o payload traz os dados do usuário e a assinatura garante a integridade. O front-end não deve confiar cegamente no payload do JWT, pois ele pode ser lido (embora não alterado sem invalidar a assinatura).

Armazenamento do token

No front-end, temos duas opções principais para armazenar o token: localStorage e sessionStorage. Ambos armazenam dados no navegador, mas o primeiro persiste após o fechamento da aba, enquanto o segundo não.

É importante lembrar que qualquer dado armazenado no navegador pode ser acessado por scripts maliciosos se houver vulnerabilidades XSS. Por isso, o ideal é evitar armazenar tokens em locais acessíveis via JavaScript sempre que possível. Quando a aplicação permitir, cookies HTTP-only são uma opção mais segura.

Fluxo de autenticação no React

O fluxo básico de autenticação no React segue os seguintes passos:

1. O usuário preenche o formulário de login;
2. A aplicação envia as credenciais para a API de autenticação;
3. A API responde com um token JWT em caso de sucesso;
4. O front-end armazena esse token e atualiza o estado da aplicação para refletir que o usuário está logado;
5. Em chamadas futuras, o token é incluído nos headers das requisições.

Protegendo rotas com autenticação

Para proteger rotas no React, podemos usar uma abordagem com componentes de rota privada. Esses componentes verificam se o usuário está autenticado antes de renderizar o conteúdo. Se não estiver, redirecionam para a página de login.

É possível usar bibliotecas como `react-router-dom` para implementar isso com clareza e manutenção facilitada. O componente que faz a verificação do token pode ficar isolado e ser reutilizado em várias partes da aplicação.

Logout e expiração de sessão

Para realizar o logout, basta remover o token armazenado e redirecionar o usuário para a tela de login. Além disso, tokens JWT geralmente têm um tempo de expiração, configurado no payload. O front-end deve estar preparado para lidar com isso, detectando quando a sessão expirou e solicitando um novo login.

Boas práticas de autenticação

- Nunca exponha dados sensíveis no payload do JWT;
- Evite deixar o token acessível via JavaScript se não for necessário;
- Use HTTPS sempre para proteger a comunicação com a API;
- Valide sempre o token no back-end, mesmo que ele pareça válido no front-end;
- Planeje fluxos de renovação de token (refresh tokens) quando aplicável.

Conclusão

Compreender e implementar autenticação corretamente é essencial para qualquer aplicação profissional. Neste capítulo, vimos os fundamentos desse processo no front-end, abordando tanto a parte técnica quanto as boas práticas de segurança.

Na próxima aula, vamos continuar evoluindo nossa aplicação com foco em arquitetura e organização do código. Seguimos!

Capítulo 5.1: Projeto Guiado - Mini App Completo

Neste capítulo, vamos colocar em prática boa parte do que exploramos até aqui, construindo um mini app completo em React. Essa etapa é essencial para consolidar o conhecimento, exercitar a organização do código, entender o fluxo de dados e aplicar os conceitos de componentes, estado, efeitos colaterais e comunicação com APIs.

Propósito do projeto guiado

O objetivo é construir uma aplicação funcional, com fluxo completo de consumo de API, exibição de informações e interação com o usuário. A proposta não é apenas montar a interface, mas sim estruturar a lógica da aplicação de maneira coesa e escalável.

Neste mini app, vamos trabalhar com chamadas a uma API pública de filmes. A aplicação deverá:

- Listar os filmes em destaque;
- Permitir busca de filmes pelo nome;
- Exibir detalhes de um filme selecionado.

Estrutura inicial e boas práticas

Ao iniciar o projeto, organizamos a estrutura de pastas de forma clara, separando componentes, estilos, serviços e páginas. É importante manter essa separação desde o início para facilitar manutenção e expansão futura.

Criamos componentes reutilizáveis para a interface, como cartões de filmes e campos de busca. Também estruturamos os serviços de comunicação com a API em arquivos separados, centralizando as chamadas HTTP e evitando repetição de código.

Gerenciamento de estado e ciclo de vida

Utilizamos `useState` para controlar o estado da aplicação: dados dos filmes, termos de busca, seleção do usuário, entre outros. Também aplicamos o `useEffect` para carregar os filmes em destaque assim que o componente principal é montado.

Esse uso integrado de estado e efeitos colaterais é um ponto central do React moderno. A aplicação se torna reativa, respondendo às mudanças no estado e atualizando a interface conforme necessário.

Busca e filtragem

Ao digitar no campo de busca, capturamos o valor com um evento de input e armazenamos no estado. A cada alteração, disparamos uma nova chamada à API com o termo informado. Essa abordagem permite uma experiência fluida, com resultados sendo exibidos em tempo real.

Para evitar chamadas excessivas, abordagens como debounce podem ser aplicadas futuramente, mas neste ponto o foco é entender o ciclo completo de captura, atualização de estado e renderização com base na resposta.

Detalhamento e renderização condicional

Ao clicar em um cartão de filme, atualizamos o estado com o filme selecionado. A interface então passa a renderizar os detalhes completos desse filme, incluindo informações como título, sinopse, data de lançamento e imagem.

Essa lógica de renderização condicional é típica em aplicações reais: em vez de navegar para outra página, é comum reaproveitar o componente e apenas alterar o conteúdo exibido com base no estado atual.

Responsividade e UX

Durante a construção do mini app, aplicamos conceitos de responsividade para garantir uma boa experiência em diferentes tamanhos de tela. Também cuidamos da experiência do usuário exibindo mensagens de loading enquanto os dados são carregados e mensagens de erro quando há falhas na chamada da API.

Esses pequenos detalhes fazem toda a diferença para entregar uma aplicação robusta e pronta para produção.

Conclusão

Com esse mini app guiado, consolidamos os principais conceitos do desenvolvimento com React: componentes reutilizáveis, gerenciamento de estado, efeitos colaterais, comunicação com API, renderização condicional e boas práticas de arquitetura.

Na próxima parte, vamos aprofundar essa aplicação com mais funcionalidades, melhor organização e preparação para cenários mais próximos de um ambiente real. Seguimos para a aula 5.2!

Capítulo 5.2: Projeto Guiado - Mini App Completo **(continuação)**

Dando continuidade ao projeto iniciado na aula anterior, neste capítulo vamos aprimorar o mini app React com mais funcionalidades, organização de páginas, controle de navegação e preparação para uma arquitetura mais próxima da realidade de projetos profissionais.

Roteamento com React Router

Um dos principais avanços nesta etapa é a introdução do react-router-dom, uma biblioteca que permite controlar a navegação entre páginas na aplicação React. Ao configurar as rotas, criamos uma navegação mais fluida e organizada, substituindo a renderização condicional por caminhos bem definidos.

Definimos rotas como:

- `/`: página principal com lista de filmes;
- `/filme/:id`: página de detalhes do filme;
- `*`: rota coringa para tratar erros de navegação (página não encontrada).

Essa estrutura melhora a legibilidade do código e torna a aplicação mais alinhada às expectativas dos usuários modernos.

Páginas como componentes

Organizamos cada rota como um componente independente dentro da pasta `pages/`. Essa abordagem facilita a manutenção e permite separar claramente as responsabilidades de cada parte da aplicação.

Cada página se torna um componente React que pode conter lógica de estado, efeitos colaterais e renderização específica. Isso nos ajuda a manter o código modular e reutilizável.

Navegação programática

Para permitir que a aplicação redirecione o usuário dinamicamente (como ao clicar em um cartão de filme), utilizamos o hook `useNavigate` do `react-router-dom`. Com ele, é possível redirecionar o usuário para a rota correta com base em eventos da interface.

Essa navegação programática é essencial para fluxos mais interativos e contextuais, como detalhes de itens, autenticação ou redirecionamentos condicionais.

Consumo de API com ID dinâmico

Na página de detalhes do filme, utilizamos o hook `useParams` para capturar o ID do filme diretamente da URL. Com esse valor, fazemos uma nova chamada à API para buscar os dados específicos do filme selecionado.

Esse é um padrão comum em aplicações React: rotas dinâmicas que permitem identificar e carregar dados baseados na URL atual.

Tratamento de erros e loading por página

Com a separação por páginas, também fica mais fácil aplicar loading e tratamento de erros localmente em cada rota. Isso melhora a experiência do usuário e evita comportamentos indesejados caso uma API falhe ou o dado não esteja disponível.

Estilização e componentização

Mantivemos a abordagem de componentes reutilizáveis para elementos visuais como cards e formulários. Isso torna a estilização mais coesa e facilita a evolução da aplicação. Utilizamos também boas práticas como separação de responsabilidades entre visual e lógica.

Preparando para a produção

Com a aplicação funcionando, começamos a refletir sobre como empacotar e hospedar. Falamos sobre o processo de build, que gera os arquivos otimizados para produção, e sobre alternativas de hospedagem, como o Vercel, que se integra bem com projetos baseados em React.

Conclusão

Com este capítulo, encerramos a construção do mini app completo em React. Foi um processo guiado e prático que envolveu conceitos fundamentais e boas práticas que você deve levar para qualquer projeto.

Nas próximas disciplinas, vamos nos aprofundar em temas mais específicos como testes, performance, acessibilidade e integração com back-end real. Mas a base está bem estruturada. Até a próxima!

Capítulo 6: Entrevistas Técnicas e Desafios Reais

Neste último capítulo, vamos abordar um aspecto essencial da vida profissional de qualquer desenvolvedor: entrevistas técnicas. Vamos falar sobre os tipos de entrevistas, como se preparar, o que é esperado, e como os desafios que enfrentamos no dia a dia estão diretamente ligados ao que aparece nos processos seletivos.

Tipos de entrevistas

Os processos seletivos para vagas de front-end geralmente incluem mais de uma etapa. Algumas das mais comuns são:

- **Triagem técnica:** perguntas objetivas ou testes rápidos para validar conhecimentos básicos;
- **Entrevista com tech lead:** conversa com foco em experiência, arquitetura, decisões de projeto e raciocínio lógico;
- **Live coding:** resolução de um problema em tempo real, com compartilhamento de tela;
- **Desafio técnico assíncrono:** desenvolvimento de uma pequena aplicação ou funcionalidade com prazo definido;
- **Análise de código ou pair programming:** leitura e discussão de um trecho de código junto com outro profissional.

Cada empresa tem seu estilo, mas é importante estar preparado para lidar com essas abordagens de forma equilibrada.

Como se preparar

A preparação para entrevistas é algo contínuo. O ideal é manter-se sempre praticando. Algumas recomendações são:

- **Estude estruturas de dados e algoritmos**, especialmente listas, pilhas, filas, mapas, ordenação e buscas. Mesmo em front-end, esse conhecimento é exigido;
- **Resolva desafios em plataformas como HackerRank, LeetCode e CodeSignal**;
- **Pratique live coding** com tempo cronometrado para simular o ambiente real;
- **Monte projetos próprios**, mesmo simples, mas que mostrem boas práticas, componentização e arquitetura;
- **Revise conceitos fundamentais de JavaScript, React, HTML, CSS e APIs**;
- **Treine explicação verbal de soluções**, pois clareza na comunicação é avaliada.

O que as empresas estão buscando

Mais do que apenas saber escrever código, as empresas buscam:

- Capacidade de resolver problemas;
- Raciocínio lógico e organização de ideias;
- Clareza ao comunicar soluções;
- Código limpo, legível e bem estruturado;
- Domínio da stack proposta (ex: React, TypeScript, APIs);
- Experiência com testes, Git e integração com back-end;
- Atenção a requisitos não funcionais como performance, acessibilidade e responsividade.

Ter experiência prática ajuda, mas mesmo que você ainda não tenha atuado profissionalmente, é possível se destacar mostrando projetos bem construídos e capacidade de aprendizado.

Como enfrentar o desafio com segurança

Ao receber um desafio, siga essas boas práticas:

- Leia com atenção todos os requisitos antes de começar;
- Planeje a solução antes de codar;
- Escolha um escopo claro e foque em entregar bem o essencial;
- Documente o projeto (README com instruções e decisões);
- Escreva testes se for solicitado;
- Use boas práticas de Git e organize os commits;
- Não tente impressionar com algo que você não domina.

A entrevista é uma troca. Mostre o que você sabe com segurança e esteja aberto para aprender com o processo.

Conclusão

Com este capítulo encerramos o módulo de Fundamentos de Front-End com React. Além dos conhecimentos técnicos, buscamos desenvolver a mentalidade necessária para atuar como desenvolvedor front-end moderno.

Agora você tem base para evoluir com ferramentas, bibliotecas e desafios mais avançados. Use esse conhecimento como alicerce e siga praticando, construindo e se desafiando.

Obrigado!

